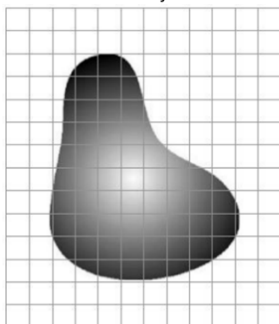
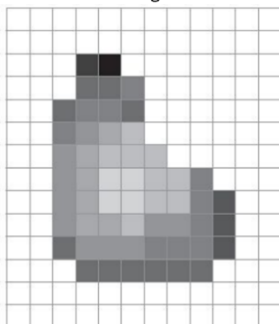


Chương 1: TỔNG QUAN VỀ THỊ GIÁC MÁY TÍNH VÀ XỬ LÝ ẢNH**NỘI DUNG**

1. Thị giác Máy tính và Xử lý Ảnh là gì?	2
2. Các vấn đề liên quan Thị giác máy tính (Computer Vision)	4
2.1. Lịch sử phát triển của Thị giác Máy tính (Computer Vision)	4
2.2. Các tác vụ của Thị giác máy tính	5
2.3. Các ứng dụng thực tế của Thị giác máy tính	5
2.4. Ưu điểm và nhược điểm của Thị giác máy tính	7
2.5. Xu hướng và Sự phát triển tương lai của Thị giác Máy tính	7
3. Các vấn đề liên quan Xử lý ảnh (Image Processing)	8
3.1. Các khái niệm cơ bản về ảnh	8
3.1.1. Ảnh là gì?	8
3.1.2. Cấu trúc của một ảnh số	8
3.1.3. Phân loại ảnh số	10
3.1.4. Các không gian màu	10
3.1.5. Các thuật ngữ khác về ảnh	14
3.1.6. Các ví dụ về ảnh số	15
3.2. Xử lý ảnh là gì?	16
3.3. Ứng dụng của xử lý ảnh	17
3.4. Xử lý ảnh và các ngành liên quan	17
3.5. Các thuật toán xử lý ảnh cơ bản	17
4. Các kỹ thuật điển hình của xử lý ảnh và thị giác máy tính	18
5. Các thư viện Python hỗ trợ cho Thị giác máy tính và Xử lý ảnh	19
6. Phụ lục	20
6.1. Các loại dữ liệu	20
6.2. Các mục tiêu tối thượng của AI	22
6.3. Some Machine Learning, Deep Learning terms	23
6.4. Các công cụ & thư viện của Python & hỗ trợ image processing	25

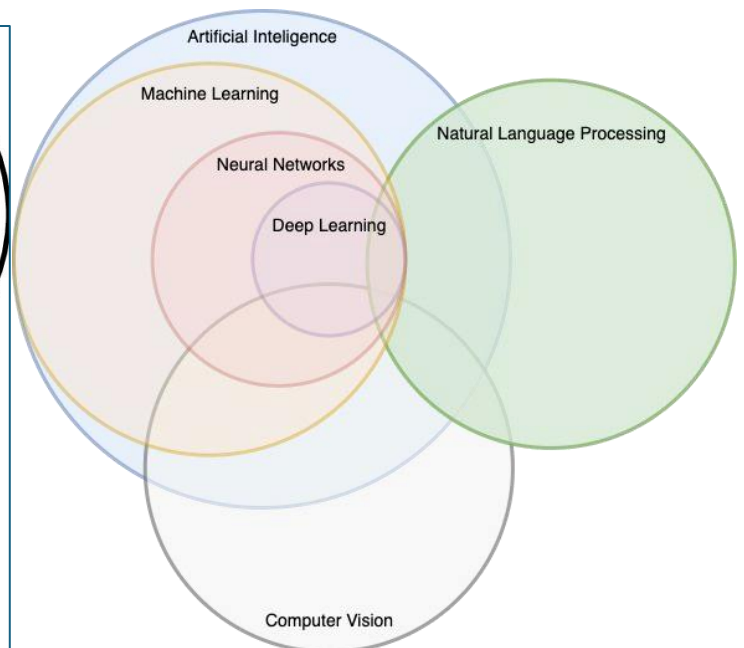
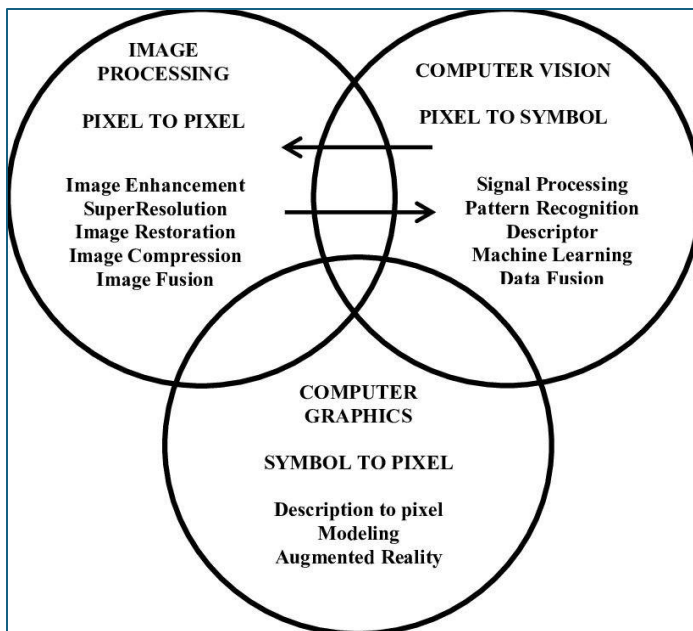
1. Thị giác Máy tính và Xử lý Ảnh là gì?

- Thị giác máy tính (Computer Vision) và xử lý ảnh (Image processing) là hai lĩnh vực liên quan chặt chẽ trong **khoa học máy tính**, tập trung vào việc **làm cho máy tính có khả năng "nhìn" và hiểu được thế giới xung quanh thông qua hình ảnh và video**.

	Xử lý ảnh (Image processing)	Thị giác máy tính (Computer Vision)																																																																																	
Mục tiêu	- Tập trung vào việc biến đổi hình ảnh số để cải thiện chất lượng hình ảnh, trích xuất thông tin hoặc chuẩn bị hình ảnh cho các bước xử lý tiếp theo. - Thao tác, chỉnh sửa các khía cạnh trực quan của hình ảnh. Real Object  Image 	- Hướng tới việc hiểu nội dung của hình ảnh, giống như cách con người nhìn và hiểu thế giới xung quanh. - Focuses on enabling computers to identify and understand objects and people in images and videos. - Trích xuất thông tin chi tiết/tri thức từ hình ảnh và video.  What we see <table><tr><td>0</td><td>3</td><td>2</td><td>5</td><td>4</td><td>7</td><td>6</td><td>9</td><td>8</td></tr><tr><td>3</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr><tr><td>2</td><td>1</td><td>0</td><td>3</td><td>2</td><td>5</td><td>4</td><td>7</td><td>6</td></tr><tr><td>5</td><td>2</td><td>3</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td><td>3</td><td>2</td><td>5</td><td>4</td></tr><tr><td>7</td><td>4</td><td>5</td><td>2</td><td>3</td><td>0</td><td>1</td><td>2</td><td>3</td></tr><tr><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td><td>3</td><td>2</td></tr><tr><td>9</td><td>6</td><td>7</td><td>4</td><td>5</td><td>2</td><td>3</td><td>0</td><td>1</td></tr><tr><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr></table> What a computer sees	0	3	2	5	4	7	6	9	8	3	0	1	2	3	4	5	6	7	2	1	0	3	2	5	4	7	6	5	2	3	0	1	2	3	4	5	4	3	2	1	0	3	2	5	4	7	4	5	2	3	0	1	2	3	6	5	4	3	2	1	0	3	2	9	6	7	4	5	2	3	0	1	8	7	6	5	4	3	2	1	0
0	3	2	5	4	7	6	9	8																																																																											
3	0	1	2	3	4	5	6	7																																																																											
2	1	0	3	2	5	4	7	6																																																																											
5	2	3	0	1	2	3	4	5																																																																											
4	3	2	1	0	3	2	5	4																																																																											
7	4	5	2	3	0	1	2	3																																																																											
6	5	4	3	2	1	0	3	2																																																																											
9	6	7	4	5	2	3	0	1																																																																											
8	7	6	5	4	3	2	1	0																																																																											
Đầu vào/ Đầu ra	Cả đầu vào và đầu ra đều là hình ảnh .	Đầu vào có thể là cả ảnh và video. Đầu ra có thể là một sự diễn giải (thông tin) , thường không phải là dạng hình ảnh .																																																																																	
Phạm vi	Các thao tác cấp thấp (low-level) tác động trực tiếp đến các điểm ảnh (pixel) trong ảnh.	Mang tính bao quát và toàn diện hơn.																																																																																	
Phương pháp	Sử dụng các thao tác đơn giản hoặc trực tiếp hơn.	Sử dụng các thuật toán và kỹ thuật phức tạp.																																																																																	
Các tác vụ điển hình	Cải thiện chất lượng: Tăng cường độ tương phản, khử nhiễu, điều chỉnh màu sắc. Phân tích hình ảnh: <i>Tính toán các đặc trưng hình ảnh</i> như độ sáng, độ tương phản, kết cấu. Xử lý trước: <i>Chuẩn bị hình ảnh</i> cho các thuật toán nhận dạng vật thể, phân loại hình ảnh.	Nhận dạng vật thể: <i>Xác định các đối tượng có trong hình ảnh</i> (ví dụ: người, xe, mặt). Phân loại hình ảnh: <i>Gán nhãn</i> cho hình ảnh dựa trên nội dung (ví dụ: ảnh phong cảnh, ảnh chân dung). Theo dõi vật thể: <i>Theo dõi chuyển động</i> của các đối tượng trong video. Tái tạo 3D: Tạo ra mô hình 3D từ hình ảnh 2D.																																																																																	

Ví dụ	• Bộ lọc ảnh: Làm mờ, sắc nét, biến đổi màu sắc. • Cắt xén, xoay ảnh: Điều chỉnh kích thước và hướng của hình ảnh. • Nén ảnh: Giảm kích thước file ảnh mà vẫn giữ được chất lượng hình ảnh. • Phần mềm chỉnh sửa ảnh (như Photoshop), nâng cao chất lượng hình ảnh y tế, v.v. • ...	• Xe tự lái: Nhận dạng biển báo giao thông, người đi bộ, các phương tiện khác. • Hệ thống giám sát: Phát hiện các sự kiện bất thường trong video. • Ứng dụng thực tế ảo (AV, AR): Tương tác với môi trường ảo thông qua camera. ○ AV - Autonomous Vehicles (Xe tự lái / Phương tiện tự hành) ○ AR - Augmented Reality (Thực tế tăng cường) • ...
---------------------	--	--

- **Mối quan hệ giữa Xử lý ảnh, Thị giác máy tính & các khía cạnh khác trong Khoa học máy tính:**



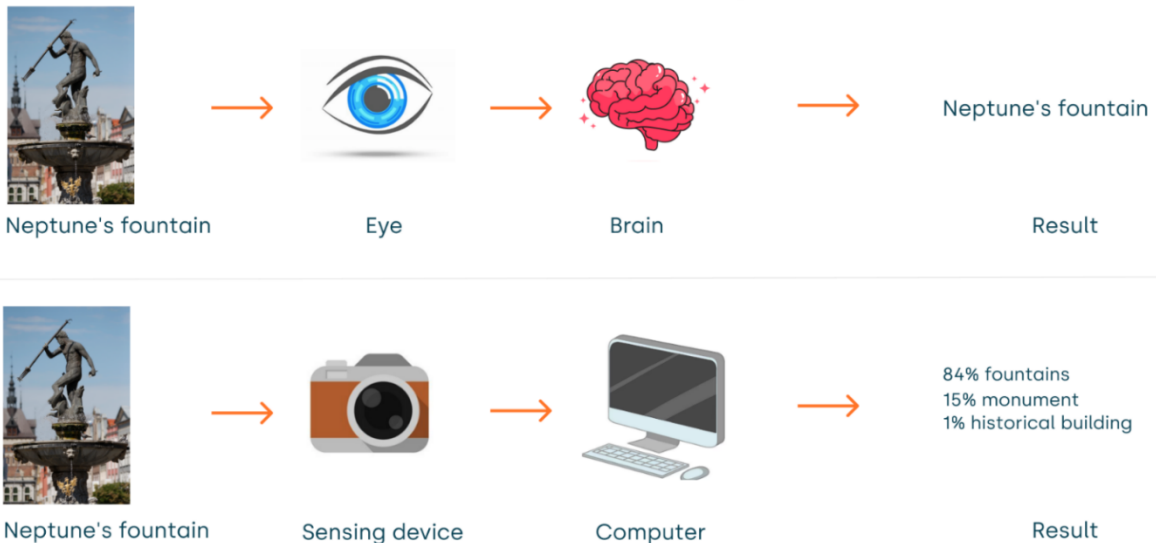
○ **Image Processing (Xử lý ảnh)**

- Nguyên tắc: Pixel to Pixel (**Đầu vào là ảnh → Đầu ra là ảnh** đã được chỉnh sửa). Hãy tưởng tượng đây là việc bạn dùng Photoshop hay các app chỉnh ảnh.
- **Image Enhancement (Tăng cường ảnh):** Làm cho ảnh đẹp hơn, dễ nhìn hơn (ví dụ: tăng độ sáng, độ tương phản).
- **Super-Resolution (Siêu phân giải):** Biến một bức ảnh mờ, độ phân giải thấp thành ảnh sắc nét, chất lượng cao, sử dụng deep learning (e.g., SRCNN, ESRGAN).
- **Image Restoration (Khôi phục ảnh):** Sửa chữa các ảnh bị hỏng, bị nhiễu, xước hoặc mờ (ví dụ: phục chế ảnh cũ).
- **Image Compression (Nén ảnh):** Làm giảm dung lượng file ảnh để dễ lưu trữ/gửi đi mà vẫn giữ được nội dung (ví dụ: ảnh JPEG).
- **Image Fusion (Ghép ảnh/Hợp nhất ảnh):** Kết hợp nhiều bức ảnh chụp cùng một cảnh (ở các điều kiện khác nhau) thành một bức ảnh hoàn chỉnh nhất.

- **Computer Vision (Thị giác máy tính)**
 - Nguyên tắc: Pixel to Symbol (**Đầu vào là ảnh → Đầu ra là thông tin/ý nghĩa**). Đây là việc dạy cho máy tính "nhìn" và "hiểu" được trong ảnh có cái gì (giống mắt và não người).
 - **Signal Processing (Xử lý tín hiệu)**: Phân tích các tín hiệu thô (màu sắc, ánh sáng) để máy tính bắt đầu "đọc" được ảnh.
 - **Pattern Recognition (Nhận dạng mẫu)**: Tìm ra các hình thù quen thuộc (ví dụ: đầu là hình tròn, đầu là khuôn mặt người).
 - **Descriptor (Mô tả đặc trưng)**: Máy tính không nhìn toàn bộ ảnh mà chọn ra các điểm chốt (góc cạnh, đường nét) để ghi nhớ đặc điểm của vật thể.
 - **Machine Learning (Học máy)**: Dạy máy tính tự học từ dữ liệu ảnh để nó ngày càng thông minh hơn trong việc nhận diện.
 - **Data Fusion (Hợp nhất dữ liệu)**: Kết hợp thông tin từ nhiều nguồn (ví dụ: camera thường + camera hồng ngoại) để máy hiểu chính xác hơn về môi trường.
- **Computer Graphics (Đồ họa máy tính)**
 - Nguyên tắc: Symbol to Pixel (**Đầu vào là thông tin/số liệu → Đầu ra là hình ảnh**). Đây là việc tạo ra hình ảnh từ con số không (giống làm phim hoạt hình 3D hoặc game). **Máy tính tự vẽ ra ảnh từ dữ liệu.**
 - **Description to pixel (Mô tả thành điểm ảnh)**: Chuyển các dòng lệnh, mô tả toán học thành hình ảnh hiển thị trên màn hình.
 - **Modeling (Tạo hình/Mô hình hóa)**: Dựng khung 3D cho vật thể trên máy tính (ví dụ: dựng nhân vật game).
 - **Augmented Reality (Thực tế tăng cường - AR)**: Vẽ các hình ảnh ảo chồng lên thế giới thật (ví dụ: game Pokemon GO hoặc filter tai thỏ trên TikTok).

2. Các vấn đề liên quan Thị giác máy tính (Computer Vision)

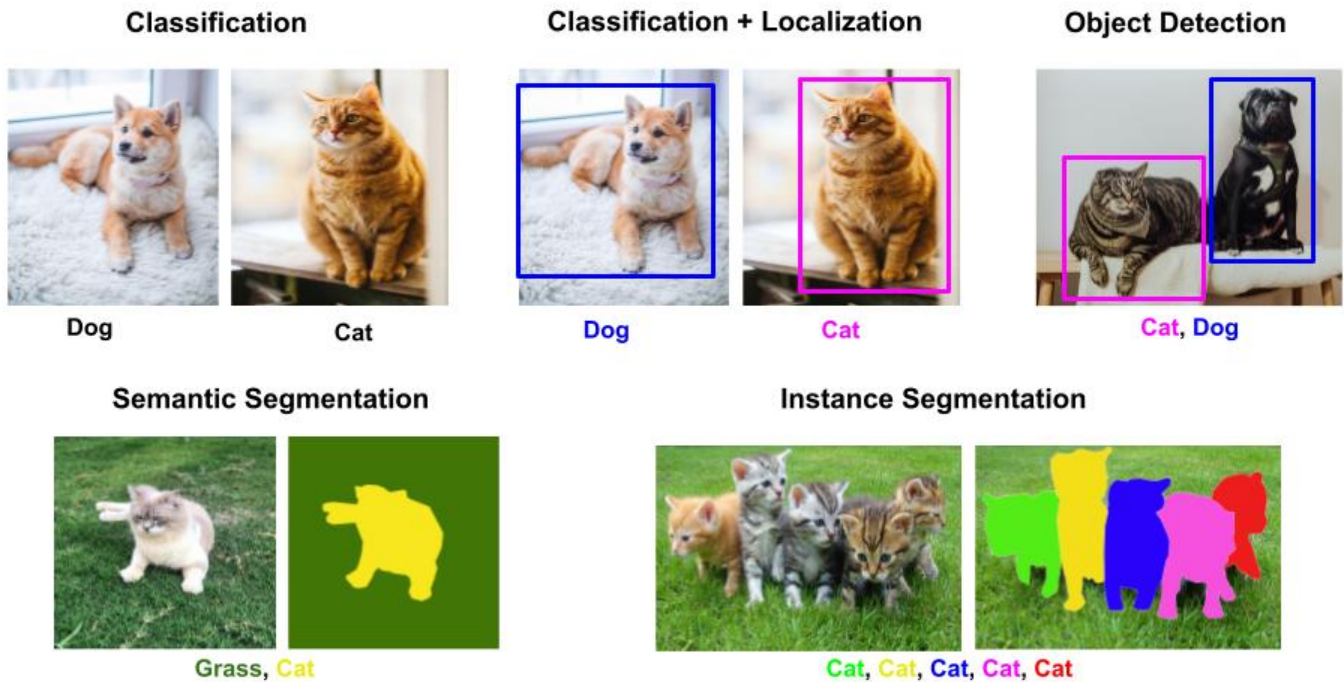
Human Vision VS Computer Vision



2.1. Lịch sử phát triển của Thị giác Máy tính (Computer Vision)

- **Giai đoạn đầu:** Những hạt giống đầu tiên – Đơn thuần đưa ra các lý thuyết, các kỹ thuật chủ yếu phụ thuộc năng lực con người. Dự án Summer Vision Project (1966) đánh dấu sự ra đời của thị giác máy tính.
- **Giai đoạn phát triển (1970-1980):** Đưa ra các thuật toán cơ bản như biến đổi Fourier, lọc, phân đoạn, các khái niệm về đặc trưng hình ảnh, mô tả hình dạng, Mạng thần kinh tích chập (CNN) do Yan LeCun giới thiệu.
- **Giai đoạn bùng nổ:** Học sâu và dữ liệu lớn - Mạng thần kinh tích chập AlexNet (2012) chứng minh sức mạnh của học sâu trong thị giác máy tính. Sau AlexNet, hàng loạt các mô hình sâu khác như VGG, ResNet, Inception được phát triển, liên tục cải thiện độ chính xác trên các bài toán thị giác máy tính.

2.2. Các tác vụ của Thị giác máy tính



2.3. Các ứng dụng thực tế của Thị giác máy tính



- **Nhận diện khuôn mặt:** Mở khóa điện thoại, xác thực danh tính, kiểm soát truy cập, theo dõi hành vi khách hàng...
 - **Face Authentication (Xác thực khuôn mặt):** Sự kết hợp của 3 lĩnh vực: **Xử lý ảnh (Image Processing) + Thị giác máy tính (Computer Vision) + Học sâu (Deep Learning)**.
 - Trong Face ID, **Feature Extraction (Trích xuất đặc trưng)** là bước quan trọng nhất để phân tích. Hệ thống không chỉ nhìn vào các bộ phận khuôn mặt (vì nhiều người có mắt/mũi giống nhau) mà ***tìm ra các đặc điểm phân biệt duy nhất***.
 - **Quy trình hoạt động (chia làm 2 giai đoạn):**

- **Giai đoạn 1: Image Processing (Xử lý ảnh)**
 - **Tiền xử lý (Preprocess):** Điều chỉnh ánh sáng, khử nhiễu, tăng độ tương phản.
 - **Phát hiện cạnh (Edge detect):** Tìm các đường nét bao quanh khuôn mặt (face contours).
 - **Căn chỉnh mặt (Face Alignment):** Xoay hoặc chỉnh lại ảnh sao cho khuôn mặt thẳng hàng để dễ xử lý.
 - **Trích xuất đặc trưng (Feature Extraction).**
- **Giai đoạn 2: Computer Vision + AI**
 - **Phát hiện điểm mốc (Facial landmark detect):** Xác định vị trí chính xác của mắt, mũi, miệng...
 - **Face Embeddings:** Sử dụng các mô hình AI (như FaceNet...) để chuyển đổi hình ảnh khuôn mặt thành **các vector số**.
 - **→ Kết quả:** So khớp (Matching) & Xác thực (Authentication) dựa trên các đặc trưng đã trích xuất.
- **Xe tự lái:** Nhận biết biển báo giao thông, vạch kẻ đường, người đi bộ, xe cộ để đưa ra quyết định lái xe an toàn...
- **Giám sát an ninh:** Phát hiện hành vi bất thường, theo dõi đối tượng, nhận dạng khuôn mặt trong các video giám sát...
- **Y tế:** Phân tích hình ảnh y tế (X-quang, MRI), hỗ trợ chẩn đoán bệnh, phẫu thuật robot... giúp bác sĩ phát hiện các dấu hiệu bệnh sớm hơn và chính xác hơn.
- **Thương mại điện tử:** Tìm kiếm hình ảnh tương tự, gợi ý sản phẩm, kiểm tra chất lượng sản phẩm, tìm kiếm sản phẩm bằng hình ảnh, thay đồ trong phiên live bán hàng bằng việc áp các hình ảnh quần áo vào người đang live...
- **Robot công nghiệp:** Nhận biết và lắp ráp các bộ phận, kiểm soát chất lượng sản phẩm...
- **Real-time AR (hiệu ứng thực tế ảo):** thử quần áo ảo, thiết kế nội thất. *Camera trên thiết bị di động sẽ quét môi trường xung quanh và hệ thống sẽ chồng các đối tượng ảo lên hình ảnh thực tế.*
- **Phân tích cảm xúc:** Đánh giá sự hài lòng của khách hàng, phân tích tâm lý, đánh giá cảm xúc của người dùng thông qua phân tích biểu cảm khuôn mặt, cử chỉ.

2.4. Ưu điểm và nhược điểm của Thị giác máy tính

Ưu điểm	Nhược điểm
• Tự động hóa hiệu quả: Tiết kiệm thời gian (giảm tác vụ lặp đi lặp lại) và nhân lực, giảm thiểu sự can thiệp của con người. Tăng năng suất do máy móc làm việc liên tục và chính xác. • Độ chính xác cao: Máy tính có thể thực hiện các phép đo và phân tích hình ảnh với độ chính xác cao hơn so với con người. Các quyết định dựa trên dữ liệu hình ảnh được đưa ra một cách khách quan. • Phân tích dữ liệu lớn: Thị giác máy tính có thể xử lý và phân tích một lượng lớn dữ liệu hình ảnh trong thời gian ngắn. Dựa trên dữ liệu được phân tích, các doanh nghiệp có thể đưa ra các quyết định kinh doanh sáng suốt hơn. • Ứng dụng đa dạng • Phát triển liên tục: Với sự phát triển của các thuật toán học sâu và phần cứng mạnh mẽ, thị giác máy tính ngày càng trở nên thông minh và hiệu quả hơn.	• Chi phí cao: Cần đầu tư vào các phần cứng (máy tính, camera, thiết bị có cấu hình cao); Các phần mềm và thuật toán thường có chi phí bản quyền cao. • Dữ liệu huấn luyện: Để huấn luyện các mô hình thị giác máy tính, cần một lượng lớn dữ liệu hình ảnh được dán nhãn chính xác. • Ánh sáng và môi trường ảnh hưởng đến kết quả: Ánh sáng, bóng đổ, và các điều kiện môi trường khác có thể ảnh hưởng đến hiệu quả của các hệ thống thị giác máy tính. • Bảo mật: Các hệ thống nhận diện khuôn mặt và phân tích hành vi có thể gây ra các vấn đề về quyền riêng tư. Thị giác máy tính hiện tại vẫn còn hạn chế trong việc hiểu ngữ cảnh và các tình huống phức tạp.

2.5. Xu hướng và Sự phát triển tương lai của Thị giác Máy tính

- Thị giác máy tính thời gian thực: Phần cứng mạnh mẽ, Thuật toán tối ưu.
- Thị giác máy tính 3D: Công nghệ cảm biến độ sâu, Ứng dụng đa dạng.
- Thị giác máy tính giải thích được: Tính minh bạch, tăng độ tin cậy.
- Học liên kết cho thị giác máy tính: Bảo vệ dữ liệu, Tăng quy mô.
- Thị giác máy tính tại cạnh mạng: Giảm độ trễ, Hoạt động ngoại tuyến.
- Thị giác máy tính trong y tế: Phân tích hình ảnh, Theo dõi sức khỏe từ xa.
- Thị giác máy tính cho robot: Robot tự động, Hợp tác giữa người và máy.
- Các vấn đề về đạo đức: Tiêu chuẩn công bằng, Bảo mật và quyền riêng tư.

3. Các vấn đề liên quan Xử lý ảnh (Image Processing)

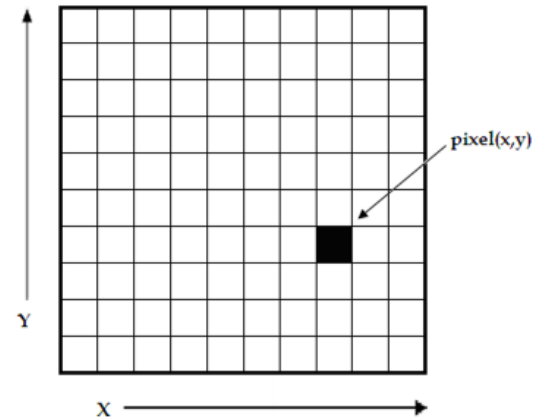
3.1. Các khái niệm cơ bản về ảnh

3.1.1. Ảnh là gì?

- **Ảnh** là **một đại diện trực quan** của một đối tượng/một cảnh vật, tạo ra bằng cách *ghi lại ánh sáng phản xạ* từ đối tượng đó.
- Trong xử lý ảnh, chúng ta thường làm việc với các **ảnh (kỹ thuật) số (digital images)**.
- *Image có mối quan hệ không gian (phân bố màu sắc theo ma trận). Video có mối quan hệ không gian và thời gian (frame by frame).*
 - Để xử lý video → trước tiên tập trung vào xử lý ảnh.
- **Ảnh là unstructure data** → Để xử lý: đưa về đặc trưng chiều rộng và chiều cao → Đưa về toán học mp Oxy.
 - Mắt người nhận biết ảnh qua cường độ ánh sáng, còn MT chỉ nhận diện qua các giá trị số.
 - Ảnh sẽ được đưa về dạng H dòng và W cột, tạo thành matrix kích thước $H \times W$.
 - Quá trình đưa dữ liệu bất kỳ về ma trận gọi là **số hóa thông tin**.
- Ảnh có thể được định nghĩa là **một hàm 2 chiều $f(x, y)$** , trong đó x và y là các tọa độ trong không gian (spatial) hoặc mặt phẳng (plane), và **độ lớn (amplitude) của hàm f** được gọi là **độ sáng (intensity)** hay **độ xám (gray level)** của ảnh tại điểm đó.

3.1.2. Cấu trúc của một ảnh số

- **Điểm ảnh (pixel):** Là **đơn vị nhỏ nhất của một ảnh số**. Mỗi pixel có một giá trị, biểu diễn **cường độ sáng** tại điểm đó. Mỗi ô vuông nhỏ trong ma trận là một pixel.
- **Ma trận điểm ảnh:** Một ảnh số được biểu diễn dưới dạng một ma trận **hai chiều**, trong đó *mỗi phần tử của ma trận tương ứng với một pixel*.
- **Độ phân giải:** **Số lượng pixel theo chiều ngang và chiều dọc** của một ảnh. Độ phân giải càng cao, ảnh càng chi tiết.
 - Một ảnh luôn có **H và W cố định**, ảnh mờ/rõ sẽ phụ thuộc cách chia số lượng “ô” trên mỗi cột & dòng.
 - **The larger Matrix, the higher resolution (ma trận càng lớn - càng nét) → Kích thước lưu trữ càng cần nhiều.**
- A 1080p image (1920×1080) means 3 matrices (1920×1080) each for R, G, and B.
- A 4K image (3840×2160) requires $24,883,200$ pixels $\times$ 3 values (R, G, B) → Large data size!

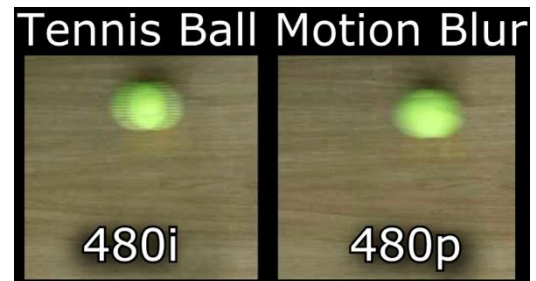


- **360p** → **640 (width) × 360 (Height)** (Standard for low-quality YouTube videos)
- **480p (SD - Standard Definition)** → **854 × 480** (DVD quality)
- **720p (HD)** → 1280 × 720
- **1080p (Full HD - FHD)** → 1920 × 1080
- **1440p (Quad HD - QHD/2K)** → 2560 × 1440
- **2160p (Ultra HD - UHD/4K)** → 3840 × 2160 (= 8,294,400 pixels ⇒ each frame will have 8.3M pixels)
- **4320p (8K UHD)** → 7680 × 4320
- **16K UHD** → 15360 × 8640

- **"p" = progressive scan (quét liên tục - lũy tiến)**, meaning all lines are displayed in each frame. Chúng ta cũng cần so sánh nó với công nghệ cũ là **"i"** (Interlaced - Quét xen kẽ). Hãy tưởng tượng bạn thuê thợ quét vôi cho bức tường.

- **Kiểu "p" (Progressive - Hiện đại):**

- Người thợ quét lần lượt từng dòng từ trên xuống dưới: Dòng 1, dòng 2, dòng 3, dòng 4, dòng 5... cho đến hết.
- Kết quả: Bức tranh hiện ra tròn vẹn, liền mạch ngay lập tức. Đây là cách màn hình máy tính, điện thoại và TV hiện nay hoạt động.



- *Con số trước "p" chỉ số lượng dòng ngang. Chữ "p" khẳng định cách vẽ các dòng đó là vẽ **liền một mạch**.*

- **Kiểu "i" (Interlaced - Cũ, ví dụ 1080i):**

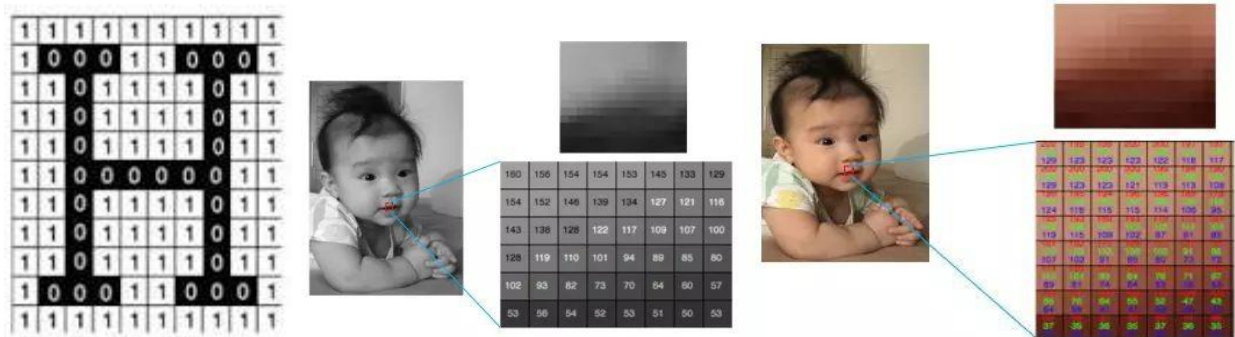
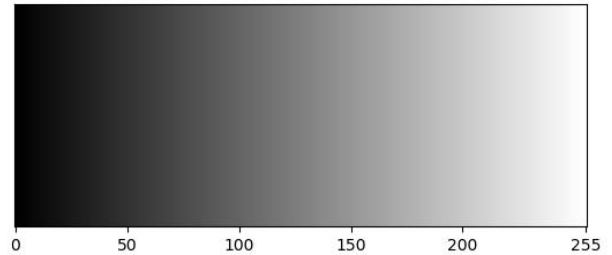
- Người thợ quét "nhảy cóc". Lần 1 anh ta chỉ quét các dòng lẻ (1, 3, 5, 7...). Lần 2 anh ta quay lại quét các dòng chẵn (2, 4, 6, 8...).
- Kết quả: Mắt người bị đánh lừa là hình ảnh liền mạch, nhưng thực chất là 2 nửa hình ảnh ghép lại.

- **Độ sâu màu: Số lượng bit sử dụng để biểu diễn màu sắc của một pixel.** Độ sâu màu càng lớn, màu sắc càng phong phú ⇒ Càng nhiều bit - càng phong phú → nhiều giá trị màu hơn.

Độ sâu màu	Số màu có thể biểu diễn	Mô tả
1-bit (Đen trắng)	2 màu (0,1)	Chỉ có 2 màu: đen và trắng
8-bit (Grayscale)	256 màu	Ảnh xám, có 256 mức từ đen → trắng
24-bit (True Color RGB)	16.7 triệu màu	Màu sắc tự nhiên, phổ biến trên màn hình
30-bit (Deep Color HDR)	1.07 tỷ màu	Dải màu rộng hơn, màu sâu và mượt hơn
48-bit (HDR, chuyên nghiệp)	281 nghìn tỷ màu	Dùng trong nhiếp ảnh, đồ họa chuyên sâu

3.1.3. Phân loại ảnh số

- Tùy vào giá trị dùng để biểu diễn điểm ảnh, có thể chia ảnh số thành 2 loại: **ảnh đen trắng và ảnh màu**.
- Ảnh đen trắng.**
 - Thực tế ảnh đen trắng lại được phân thành 2 loại khác nhau là **ảnh nhị phân (binary/black-white image)** và **ảnh xám (grayscale image)**.
 - Giá trị mỗi pixel** của **ảnh nhị phân** chỉ có giá trị **0 (black)** hoặc **1 (white)**. Ứng dụng Text Scans, QR code, barcodes...
 - Giá trị mỗi pixel** của **ảnh xám** là một số nguyên nằm trong khoảng từ **0 đến 255** - là dải màu từ đen về trắng, không phải 2 màu đen trắng độc lập – thể hiện cường độ màu sắc tại pixel đó. Trong đó **mức 0** biểu diễn cho **mức cường độ (intensity) đen nhất** và **255** biểu diễn cho **mức cường độ sáng nhất**.
- Ảnh màu.** Tùy theo từng trường hợp mà ảnh màu được biểu diễn bằng những **không gian màu** khác nhau.



3.1.4. Các không gian màu

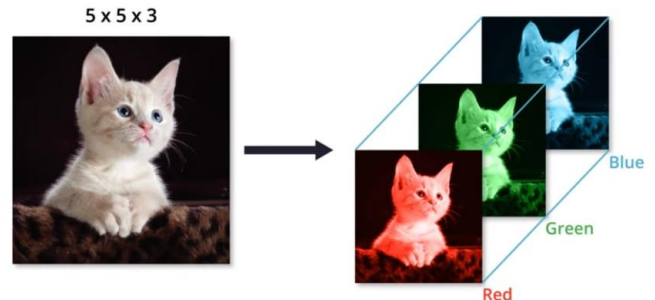
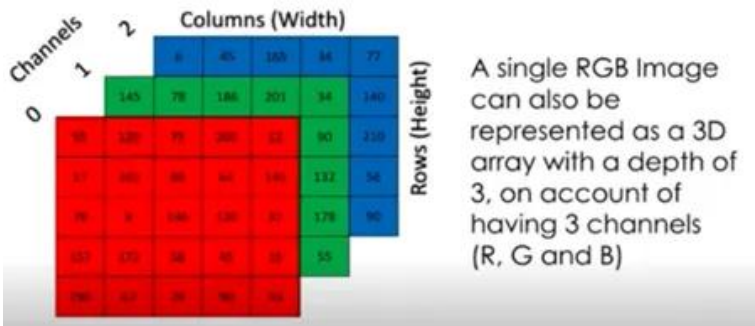
- Không gian màu chính là **mô hình toán học** dùng để **mô tả các màu sắc** trong thực tế dưới dạng số học.
- Không có mô hình nào có thể biểu diễn đầy đủ mọi khía cạnh của màu. Do đó phải **sử dụng những mô hình khác nhau để mô tả những tính chất được nhận biết khác nhau của màu**.
- RGB (True Color):** Mô hình màu dựa trên ba màu cơ bản: **Đỏ (Red), Lục (Green), Lam (Blue)**.
 - Mô hình màu phổ biến nhất, được sử dụng rộng rãi trong các thiết bị hiển thị.
 - Màu sắc của mỗi pixel được xác định bởi giá trị RGB tương ứng. Mỗi pixel giữ 1 giá trị RGB, typically contains **3 kênh màu RGB**.
 - Mỗi kênh màu được mã hóa bằng 1 byte (8 bits)** → total of 24 bits per pixel → ta có ảnh 24 bit màu
 - Giá trị mỗi kênh nằm trong đoạn $[0, 255]$ → mã hóa được tất cả $255 \times 255 \times 255 = 16.581.375$ màu, hay thường gọi là **16 triệu màu**.



- Nếu bạn pha trộn các giá trị này, bạn sẽ tạo ra các màu khác. Ví dụ:
 - Màu Đen:** rgb(0, 0, 0) (Tắt hết đèn).
 - Màu Trắng:** rgb(255, 255, 255) (Bật sáng hết cỡ cả 3 đèn).
 - Màu Vàng:** rgb(255, 255, 0) (Pha Đỏ + Xanh lá).
 - Màu Xám:** rgb(128, 128, 128) (Cả 3 màu đều ở mức trung bình).

- Vùng màu Đỏ (Red Block) - Đỏ thuần khiết**
 - Mã màu:** rgb(255, 0, 0)
 - Kênh **Red (r)** đặt ở mức cao nhất (255).
 - Kênh **Green (g)** và **Blue (b)** đặt ở mức thấp nhất (0).
- Vùng màu Xanh lá (Green Block) - Xanh lá thuần khiết**
 - Mã màu:** rgb(0, 255, 0)
 - Kênh **Green (g)** đặt ở mức cao nhất (255).
 - Kênh **Red (r)** và **Blue (b)** đặt ở mức thấp nhất (0).
- Vùng màu Xanh dương (Blue Block) - Xanh dương thuần khiết**
 - Mã màu:** rgb(0, 0, 255)
 - Kênh **Blue (b)** đặt ở mức cao nhất (255).
 - Kênh **Red (r)** và **Green (g)** đặt ở mức thấp nhất (0).

- A color image (RGB image) = **3D matrix**, consisting of **three separate 2D matrices**.

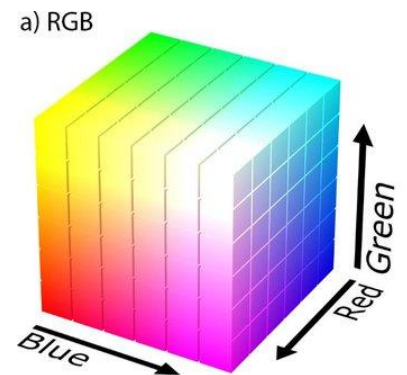
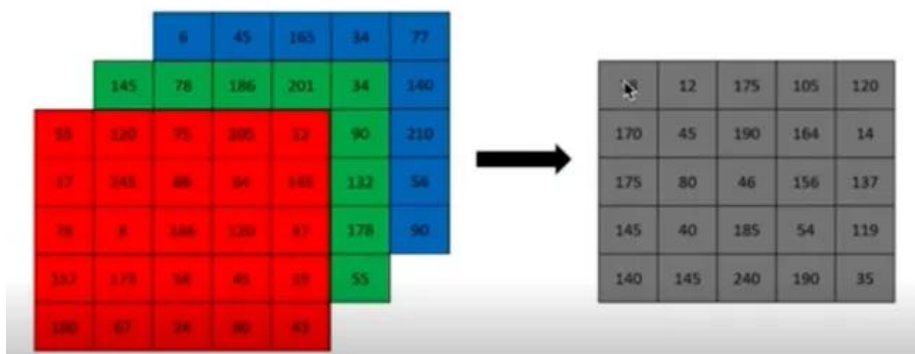


- Công thức chuyển đổi từ **RGB sang grayscale:**

$$\text{Độ sáng} = 0.2989R + 0.5870G + 0.11140B$$

Với R, G, B là các ma trận số.

Các hệ số có thể được làm tròn thành **0.3, 0.59 và 0.11**.



❖ **True Color và Deep Color là gì?**

	True Color (Màu thực - 24 bit)	Deep Color (Màu sâu - 30 bit trở lên)
Định nghĩa	Là tiêu chuẩn phổ biến nhất hiện nay (web, màn hình máy tính văn phòng, điện thoại thông thường). Nó được gọi là "True" vì mắt người bình thường khó phân biệt được các dải màu riêng lẻ ở mức này, hình ảnh trông đã đủ "thực".	Là các chuẩn cao cấp hơn, có khả năng hiển thị hàng tỷ màu. Mục đích chính là để xóa bỏ hiện tượng gãy màu (color banding) khi hiển thị các vùng chuyển màu mượt mà (như bầu trời, mặt nước).
Tổng số Bit (toàn ảnh)	24-bit	30-bit, 36-bit, 48-bit
Số Bit mỗi kênh (R,G,B)	8-bit mỗi kênh (bpc - bits per channel)	10-bit, 12-bit, 16-bit mỗi kênh
Số lượng màu tối đa	≈ 16.7 triệu màu ($2^8 \times 2^8 \times 2^8$)	1 tỷ màu (30-bit) đến 281 nghìn tỷ màu (48-bit)
Dải giá trị (mỗi kênh)	0 - 255	0 - 1023 (10-bit) hoặc lớn hơn
Hiện tượng thị giác	Có thể bị Color Banding (gãy màu/vết sọc) khi nhìn các vùng chuyển màu rộng (gradient).	Mịn màng tuyệt đối , không thấy vết gãy giữa các tầng màu.
Ứng dụng phổ biến	Ảnh JPEG, Video Youtube, Web, Màn hình văn phòng, TV thường.	Chỉnh sửa ảnh HDR, In ấn chuyên nghiệp, Y tế (DICOM), Phim điện ảnh (DCP), Kỹ xảo (VFX).

- **RGBA color** là phần mở rộng của giá trị màu RGB, với thành phần bổ sung là chỉ số opacity quy định độ mờ/độ trong suốt của màu sắc.

- **rgba(red, green, blue, alpha)**
 - Trong đó alpha là chỉ số opacity có giá trị từ 0.0 - 1.0, **giá trị càng nhỏ thì độ trong suốt càng nhiều**.

- **HSV:** Mô hình màu dựa trên ba thuộc tính:

- **Hue:** sắc độ, có giá trị từ **0 đến 360°**. Changing hue can shift an image from warm to cool tones.
 - **Saturation:** độ bão hòa - Điều chỉnh **độ đậm nhạt của màu sắc** - Mức độ tinh khiết của màu, nằm trong đoạn [0, 1], trong đó S = 1 là màu tinh khiết nhất).
 - **Value:** độ sáng, Intensity, Lightness, cũng có giá trị dao động trong đoạn [0, 1], trong đó V = 0 là hoàn toàn tối (đen), V = 1 là hoàn toàn sáng.
 - Không gian màu này còn có tên khác là **HSI (intensity)**, **HSL (lightness)**.

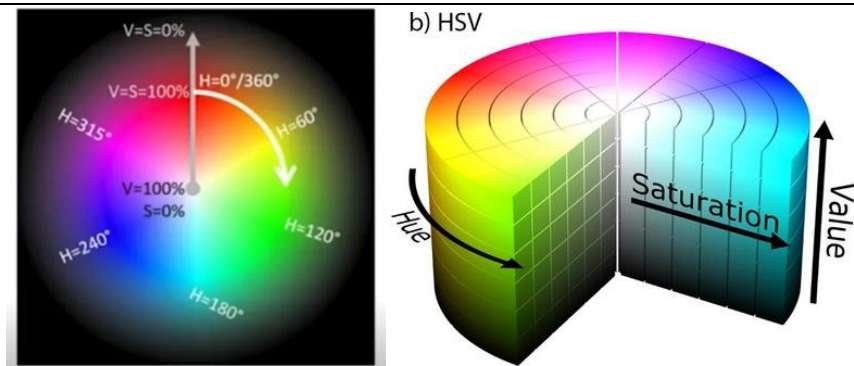
rgba(0, 0, 255, 0.2)

rgba(0, 0, 255, 0.4)

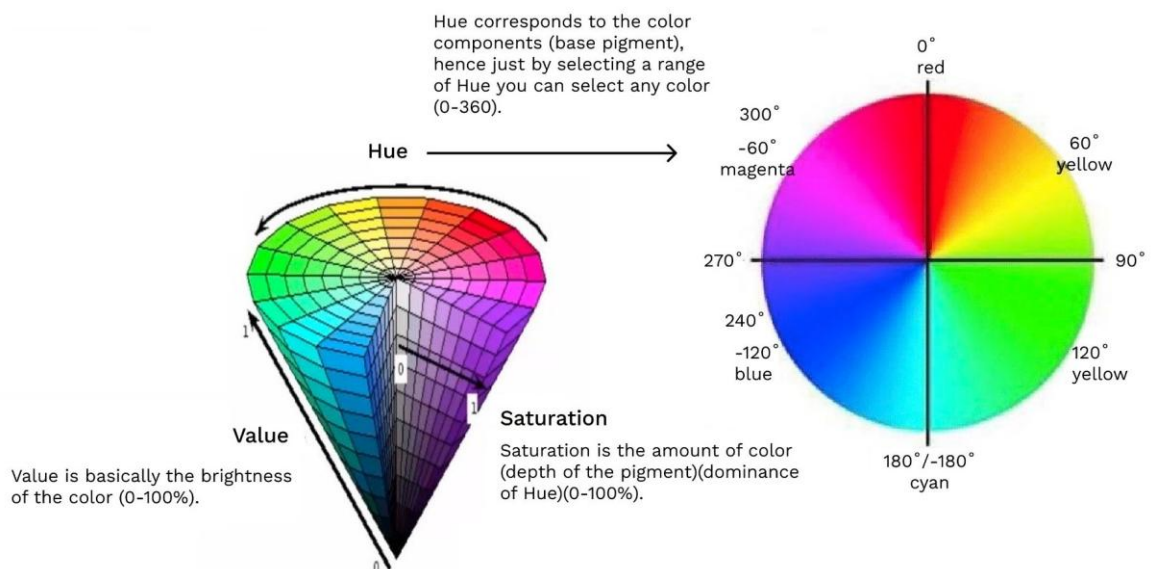
rgba(0, 0, 255, 0.6)

rgba(0, 0, 255, 0.8)

rgba(0, 0, 255, 1)



HSV (Hue, Saturation and Value)



- **CMYK:** Mô hình màu sử dụng trong **in ấn**, dựa trên bốn màu mực: *Cyan, Magenta, Yellow, Black*.



3.1.5. Các thuật ngữ khác về ảnh

- **Chỉnh màu**

- **Color Correction (Chỉnh màu kỹ thuật):** Là bước "sửa lỗi". Mục tiêu là đưa màu sắc của ảnh/video về trạng thái **tự nhiên nhất**, đúng với mắt nhìn thực tế (ví dụ: sửa da bị ám xanh, sửa ảnh quá tối).
- **Color Grading (Chỉnh màu nghệ thuật):** Là bước "sáng tạo". Sau khi ảnh đã đúng màu, ta chỉnh tiếp để tạo ra **cảm xúc hoặc phong cách riêng** (ví dụ: làm màu phim cũ, màu lạnh lẽo kinh dị, màu ám áp lãng mạn).

- **Ánh Sáng & Tương Phản**

- **Luminance (Độ chói/Độ sáng):** Là thông tin về **độ sáng - tối** của bức ảnh mà **không quan tâm đến màu sắc** (giống như nhìn ảnh đen trắng). *Ví dụ:* Trong hệ màu YUV, máy tính tách riêng phần này ra để xử lý độ nét trước khi xử lý màu.
- **Exposure (Độ phơi sáng):** Mô phỏng **lượng ánh sáng đi vào camera**. **Tăng Exposure** giống như bạn mở rèm cửa ra to hơn, toàn bộ căn phòng rực sáng lên (chú ý: tăng quá đà sẽ làm mất chi tiết ở những chỗ sáng nhất - bị "cháy").
- **Brightness (Độ sáng tổng thể):**
 - Làm bức ảnh sáng lên hoặc tối đi một cách **đều đặn** (giống như tăng giảm độ sáng màn hình điện thoại).
 - *Khác với Exposure:* Brightness tác động nhẹ nhàng hơn vào vùng sáng nhất, chủ yếu **làm sáng vùng tối và trung tính**.
- **Contrast (Độ tương phản):**
 - Sự **chênh lệch giữa chỗ sáng nhất và chỗ tối nhất**.
 - **Tương phản cao:** Ảnh sắc nét, gắt, màu đen rất sâu, màu trắng rất sáng (tạo cảm giác mạnh mẽ).
 - **Tương phản thấp:** Ảnh trông mờ mờ, đục đục, xám xịt (tạo cảm giác mộng mơ hoặc buồn tẻ).
- **Light Balance (Cân bằng sáng):** Điều chỉnh để ánh sáng phân bố đều. Giúp **cứu những chỗ bị quá sáng hoặc quá tối**, làm bức ảnh hài hòa hơn.
- **Highlight (Vùng sáng/Vùng cao quang):** Là những **khuvực sáng nhất** trong ảnh (ví dụ: bầu trời, ánh đèn, vệt nắng trên mặt). *Chỉnh Highlight* → Thường là **giảm xuống** để nhìn thấy mây trên trời (tránh bị trắng xóa).
- **Shadow (Vùng tối/Vùng bóng):** Là những khu vực bị khuất sáng, tối tăm (ví dụ: bóng râm, góc nhà, hốc mắt). *Chỉnh Shadow* → Thường là **tăng lên** để nhìn rõ chi tiết đang ẩn trong bóng tối.

- **Màu Sắc (Color Balance)**

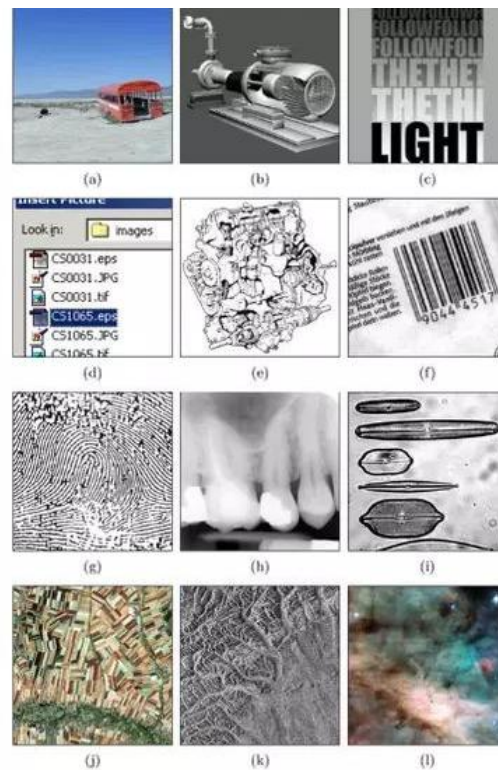
- **Temperature / Temp (Nhiệt độ màu):** Điều chỉnh **độ "nóng - lạnh"** của ảnh. **Measured in Kelvin (K)**, lower values = warm, higher values = cool.
 - **Kéo sang vàng (Warm):** Ảnh ấm áp (như nắng chiều, ánh đèn ngủ).
 - **Kéo sang xanh dương (Cool):** Ảnh lạnh lẽo (như buổi sớm, trời mưa).
- **Tint (Ám màu):** Điều chỉnh cân bằng giữa màu **Xanh lá (Green)** và **Tím hồng (Magenta)**. Thường dùng để khử lỗi màu do đèn huỳnh quang (khiến da bị ám xanh lá cây).

• Chi Tiết & Cấu Trúc

- **Sharpness (Độ sắc nét):** Làm rõ các đường viền/cạnh của vật thể. Tăng Sharpness giúp ảnh trông "bén" hơn, chữ dễ đọc hơn, lông tóc rõ hơn.
- **Definition / Clarity (Độ chi tiết / Độ rõ nét):** Tăng độ tương phản ở các chi tiết nhỏ (như nếp nhăn, vân gỗ, chất liệu vải), giúp ảnh trông nổi khối (3D) hơn mà không làm hỏng vùng sáng/tối tổng thể.
- **Gradient (Chuyển sắc):**
 - Sự thay đổi mượt mà từ màu này sang màu kia, hoặc từ sáng sang tối.
 - Ví dụ: Bầu trời lúc hoàng hôn là một gradient chuyển từ màu cam sang màu tím than. Trong chỉnh ảnh, người ta dùng Gradient để làm tối 4 góc ảnh hoặc đổi màu bầu trời.

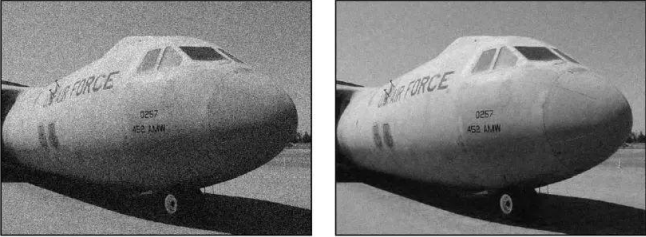
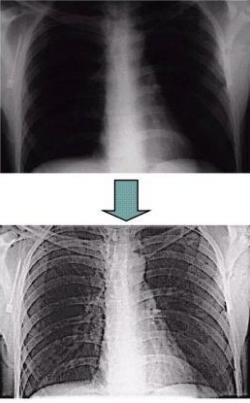
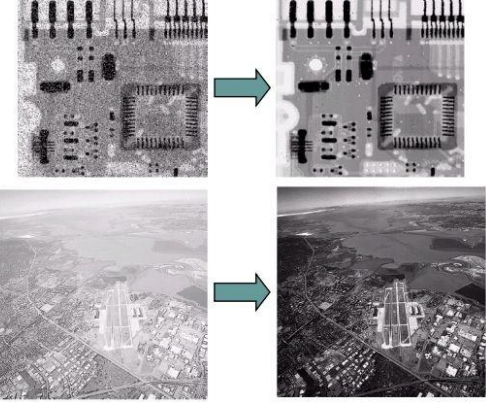
3.1.6. Các ví dụ về ảnh số

- a) Natural landscape
- b) Synthetically generated scene
- c) Poster graphic
- d) Computer screenshot
- e) Black and white illustration
- f) Barcode
- g) Fingerprint
- h) X-ray
- i) Microscope slide
- j) Satellite Image
- k) Radar image
- l) Astronomical object



3.2. Xử lý ảnh là gì?

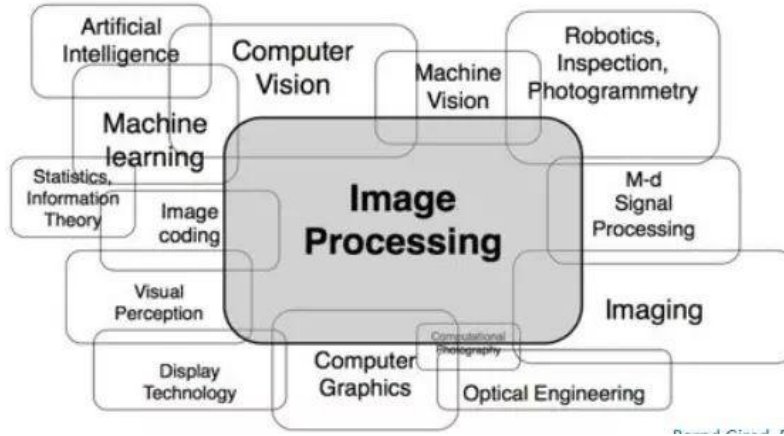
- **Xử lý ảnh** là các thuật toán thay đổi hình ảnh đầu vào để tạo hình ảnh mới.
- Là quá trình chuyển ma trận ảnh gốc sang một ma trận ảnh mới.
- Image processing task is to **apply filters that modify pixel values** of the image to create a visual effect.
- **Các ví dụ về xử lý ảnh**

Giảm nhiễu   	Điều chỉnh độ tương phản  Low Contrast Original Contrast High Contrast
Tìm cạnh 	Phân vùng 
Khôi phục ảnh 	Nén ảnh  Original, 2.1MB JPEG Compression, 308KB (15%)

3.3. Ứng dụng của xử lý ảnh

- **Xử lý ảnh y tế:** Phân tích hình ảnh X-quang, MRI để hỗ trợ chẩn đoán bệnh.
- **Xử lý ảnh vệ tinh:** Phân tích hình ảnh vệ tinh để theo dõi biến đổi môi trường, quy hoạch đô thị.
- **Xử lý ảnh công nghiệp:** Kiểm soát chất lượng sản phẩm, tự động hóa quy trình sản xuất.
- **Xử lý ảnh an ninh:** Nhận dạng khuôn mặt, theo dõi đối tượng.
- **Fingerprint Authentication (Xác thực vân tay)**
 - Là công nghệ sinh trắc học (Biometric) dựa trên nền tảng "**Xử lý ảnh**".
 - Chụp ảnh vân tay → Tăng cường chất lượng ảnh → Trích xuất đặc trưng.
 - **Quy trình hoạt động:**
 - **Thu thập ảnh (Image Acquisition):** Chụp vân tay bằng cảm biến (quang học/optical, điện dung/capacitive, hoặc siêu âm/ultrasonic).
 - **Tiền xử lý (Preprocessing):** Tăng độ tương phản, khử nhiễu và chuẩn hóa độ sáng để ảnh rõ nét hơn.
 - **Phát hiện cạnh (Edge Detection):** Xác định các đường vân nổi (ridges) và các rãnh chìm (valleys) - tạo nên các mẫu đen trắng.
 - **Nhị phân hóa & Làm mỏng (Binarization & Thinning):**
 - Chuyển ảnh về định **dạng nhị phân** (chỉ có 0 và 1).
 - Làm mỏng (Thinning): Làm nhỏ các đường vân lại để dễ dàng phân tích cấu trúc.

3.4. Xử lý ảnh và các ngành liên quan



3.5. Các thuật toán xử lý ảnh cơ bản

- **Biến đổi không gian:**
 - **Làm mờ:** **Làm mịn** hình ảnh, giảm nhiễu bằng cách tính trung bình giá trị pixel trong một vùng lân cận.
 - **Trung bình:** Mỗi pixel mới được tính bằng trung bình cộng của các pixel lân cận.
 - **Gaussian:** Sử dụng hàm Gaussian để tính trọng số cho các pixel lân cận.
 - **Tăng cường cạnh:** **Phát hiện các điểm chuyển tiếp đột ngột** của cường độ sáng trong hình ảnh.
 - **Sobel:** Tính đạo hàm xấp xỉ của cường độ sáng theo phương ngang và dọc.
 - **Canny:** Áp dụng một loạt các bước để phát hiện các cạnh mạnh mẽ và loại bỏ các cạnh yếu.
 - **Biến đổi hình học:** Thay đổi kích thước, xoay, cắt xén, biến dạng hình ảnh.

- **Biến đổi tần số: Biến đổi Fourier** → Phân tích hình ảnh thành **các thành phần tần số**, giúp loại bỏ nhiễu, nén dữ liệu.
- **Phân đoạn:**
 - **Phân đoạn ngưỡng:** Chia hình ảnh thành các vùng dựa trên một **ngưỡng cường độ**.
 - **Phân đoạn vùng:** Chia hình ảnh thành các **vùng đồng nhất** dựa trên các đặc trưng như màu sắc, kết cấu.
 - **Phân đoạn dựa trên cạnh:** Phân chia hình ảnh dựa trên **các đường viền**.
- **Mô tả hình ảnh:**
 - **Histogram:** Mô tả **phân bố cường độ sáng** của các pixel trong hình ảnh.
 - **Mô tả dựa trên mô hình:** Sử dụng các mô hình toán học (ví dụ: mô hình hình thái) để mô tả hình dạng của các đối tượng.
- **Các thuật toán khác:**
 - **Morphological:** Áp dụng các phép toán hình thái học để thay đổi hình dạng và kích thước của các đối tượng trong hình ảnh.
 - **Học sâu:** Sử dụng các mạng thần kinh để thực hiện các tác vụ xử lý ảnh phức tạp như phân loại, phát hiện đối tượng.

4. Các kỹ thuật điển hình của xử lý ảnh và thị giác máy tính

- | | |
|--|--|
| <ul style="list-style-type: none"> • Xử lý ảnh (Image processing) <ul style="list-style-type: none"> • Nén ảnh (compression) • Giảm nhiễu (noise reduction) • Nâng cao tương phản (contrast enhancement) • Lọc (filtering) • Biến đổi afin (affine transformation) • Khôi phục (restoration) | <ul style="list-style-type: none"> • Thị giác máy (Computer vision) <ul style="list-style-type: none"> • Theo vết (tracking) • Phát hiện (detection) • Phân loại (classification) • Nhận dạng (recognition) • Phân vùng ngữ nghĩa (semantic segmentation) |
|--|--|



Ảnh vào

 $f \circ g$


Ảnh ra



Ảnh vào

→



Tri thức mô tả lại ảnh

Phát hiện khuôn mặt

5. Các thư viện Python hỗ trợ cho Thị giác máy tính và Xử lý ảnh

Thư viện	Tác vụ	Ưu điểm	Nhược điểm
OpenCV: thư viện mã nguồn mở, được sử dụng trong thị giác máy tính và xử lý ảnh.	- Xử lý ảnh cơ bản, video - Phát hiện vật thể, Nhận dạng đối tượng - Trích xuất đặc trưng: SIFT, SURF, HOG, ...	- Cộng đồng người dùng lớn, nhiều tài liệu và ví dụ sẵn có. - Hiệu năng cao, phù hợp với các ứng dụng thực tế. - Hỗ trợ nhiều nền tảng, hệ điều hành và ngôn ngữ lập trình (C++, Python, Java,...)	Giao diện lập trình có thể hơi phức tạp đối với người mới bắt đầu.
Pillow: thư viện xử lý ảnh mạnh mẽ và dễ sử dụng trong Python.	- Mở và lưu trữ ảnh ở nhiều định dạng. - Thao tác trên pixel: thay đổi màu sắc, độ sáng, độ tương phản. - Thực hiện các phép biến đổi hình ảnh - Tạo ảnh mới từ đầu.	- Giao diện trực quan, dễ học và sử dụng. - Tích hợp tốt với các thư viện khác như NumPy. - Thường sử dụng như một thư viện bổ trợ cho OpenCV.	Không mạnh mẽ bằng OpenCV về các tính năng xử lý ảnh nâng cao.
scikit-image: thư viện xử lý ảnh xây dựng trên NumPy và SciPy, cung cấp các công cụ để xử lý ảnh khoa học.	- Bộ lọc: làm mờ, tăng cường cạnh, ... - Phân đoạn ảnh: chia ảnh thành các vùng có cùng đặc trưng. - Phân tích hình thái: xác định hình dạng và cấu trúc của các đối tượng trong ảnh. - Phân tích hình ảnh y sinh.	- Tích hợp tốt với hệ sinh thái SciPy, NumPy. - Thường được sử dụng trong các ứng dụng khoa học, y tế. - Cung cấp các thuật toán xử lý ảnh tiên tiến.	Cú pháp có thể hơi phức tạp hơn so với Pillow.

Tensorflow/ Keras: Keras là một API cấp cao xây dựng trên TensorFlow, giúp việc xây dựng và huấn luyện các mô hình học sâu trở nên dễ dàng hơn.	- Xây dựng các mạng thần kinh nhân tạo (ANN). - Huấn luyện các mô hình trên các tập dữ liệu lớn.	- Cộng đồng người dùng lớn, nhiều tài liệu và ví dụ sẵn có. - Hỗ trợ nhiều loại mạng thần kinh khác nhau. - Tích hợp tốt với các thư viện khác như NumPy, Pandas.	Có thể phức tạp đối với người mới bắt đầu.
Pytorch/ torchvision: torchvision là một thư viện cung cấp các tập dữ liệu, mô hình và công cụ xử lý ảnh cho PyTorch	Triển khai các mô hình trên các thiết bị khác nhau (CPU, GPU).	- Dễ sử dụng, linh hoạt. - Cộng đồng người dùng lớn và đang phát triển nhanh chóng. - Tích hợp tốt với các thư viện khác như NumPy, Pandas.	Một số tính năng có thể chưa hoàn thiện bằng TensorFlow.

- Việc lựa chọn thư viện phù hợp còn phụ thuộc vào nhiều yếu tố khác như: *ngôn ngữ lập trình, nền tảng phần cứng, yêu cầu về hiệu năng, độ phức tạp của bài toán.*

6. Phu lục

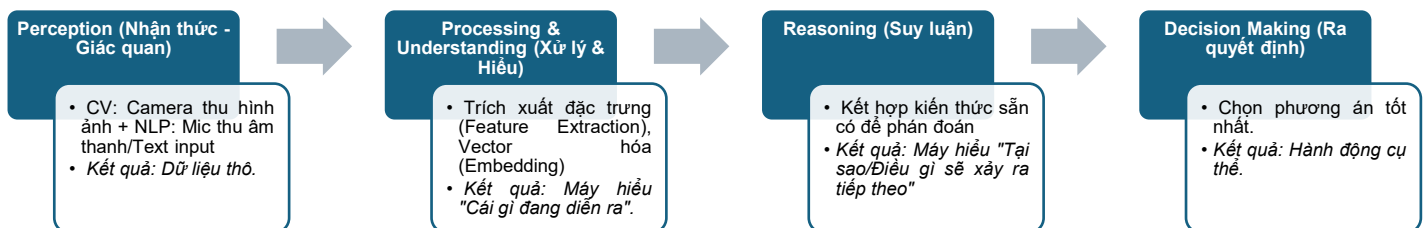
6.1. Các loại dữ liệu

- Trước đây, máy tính chủ yếu xử lý con số (Excel, Database). Ngày nay, "Dữ liệu" (Data) đã mở rộng ra thành **Multimedia Data (Dữ liệu Đa phương thức)**, với các loại như sau:
 - Visual Data (Dữ liệu thị giác):** Dữ liệu đầu vào cho lĩnh vực Computer Vision (Thị giác máy tính).
 - Các dạng tồn tại:**
 - Image (Ảnh tĩnh):** Là một ma trận các điểm ảnh (pixel).
 - Video (Ảnh động):** Thực chất là một **chuỗi các ảnh tĩnh được phát liên tục** (Frames) kèm theo âm thanh. Video bổ sung thêm yếu tố *thời gian* (temporal) vào dữ liệu ảnh.
 - 3D Point Clouds:** Dữ liệu không gian 3 chiều (thường từ cảm biến LiDAR trên xe tự lái hoặc máy quét 3D).
 - Đặc điểm:** Dữ liệu "phi cấu trúc" (Unstructured), dung lượng lớn, **chứa nhiều nhiễu.**
 - Bài toán tiêu biểu:** Nhận diện khuôn mặt, phát hiện vật thể (xe cộ, chó mèo), phân đoạn ảnh (tách nền), xe tự lái.

- **Natural Language Data (Dữ liệu ngôn ngữ tự nhiên):** Dữ liệu đầu vào cho lĩnh vực **NLP (Natural Language Processing)** và **Speech Processing**.
 - **Các dạng tồn tại:**
 - **Text (Văn bản):** Các ký tự, từ, câu, đoạn văn. Dữ liệu này **rời rạc** (discrete).
 - **Voice/Audio (Giọng nói/Âm thanh):** Dạng **sóng âm thanh** (waveforms). Để máy tính hiểu, sóng âm thường được chuyển đổi thành hình ảnh biểu đồ tần số (Spectrogram) hoặc văn bản (Speech-to-Text).
 - **Đặc điểm:** Mang tính **tuần tự (Sequential)** cao. **Ý nghĩa của một từ phụ thuộc vào các từ đứng trước và sau nó (ngữ cảnh).**
 - **Bài toán tiêu biểu:** Dịch thuật, Chatbot, Trợ lý ảo, Phân tích cảm xúc khách hàng.
- **Tabular Data & Sequences (Dữ liệu bảng & Chuỗi):** Đây là dạng dữ liệu "truyền thống" nhất nhưng vẫn cực kỳ quan trọng, thường dùng trong **Data Science** và **Machine Learning cổ điển**.
 - **Tabular Data (Dữ liệu bảng):**
 - Giống như file Excel, file .csv hoặc Database (SQL).
 - **Có cấu trúc** rõ ràng: **Hàng** (Mẫu dữ liệu) và **Cột** (Thuộc tính/Feature).
 - **Time-Series / Sequences (Chuỗi thời gian):** Dữ liệu được đo đạc liên tục theo thời gian. *Ví dụ:* Giá chứng khoán từng phút, nhịp tim bệnh nhân, dữ liệu cảm biến nhiệt độ IoT, chuỗi DNA/Protein trong sinh học.
 - Thường **ít dung lượng hơn** ảnh/video nhưng đòi hỏi độ chính xác cao về mặt thống kê.
- Các mô hình AI hiện đại không xử lý riêng lẻ nữa mà kết hợp chúng lại:
 - Multimedia Data = Visual + Language + Audio**
 - **Ví dụ 1:** Bạn gửi một bức ảnh (Visual) và hỏi "Cái này là gì?" (Text) → AI trả lời bằng giọng nói (Audio).
 - **Ví dụ 2 (YouTube):** Đề gợi ý video, AI phân tích cả hình ảnh trong video (Visual), lời thoại trong video (Language/Audio) và lịch sử xem của bạn (Tabular/Sequence).

6.2. Các mục tiêu tối thượng của AI

- **Understanding (Hiểu) - Tầng nền tảng**
 - **Định nghĩa:** Không chỉ nhận diện bề mặt (con mèo, chữ A) mà hiểu được ngữ cảnh và mối quan hệ.
 - **Trong CV:** Không chỉ biết "đây là con dao", mà hiểu "con dao đang nằm trên bàn (an toàn)" khác với "con dao đang kề cổ ai đó (nguy hiểm)".
 - **Trong NLP:** Không chỉ dịch từ sang từ, mà hiểu ẩn ý. Ví dụ câu "Trời hôm nay đẹp nhỉ" có thể là lời khen, nhưng cũng có thể là lời mỉa mai tùy ngữ cảnh.
- **Reasoning (Suy luận) - Tầng tư duy** (*Lưu ý: Trong thuật ngữ chuyên ngành, Reasoning thường được dịch là Suy luận/Tư duy logic*)
 - **Định nghĩa:** Khả năng kết nối các mảnh thông tin rời rạc để **tìm ra cái mới chưa có trong dữ liệu gốc**.
 - **Ví dụ:**
 - *Dữ liệu:* Trời mưa + Đường ướt + Xe đi nhanh.
 - *Reasoning:* Xe rất dễ bị trượt ngã → Cần giảm tốc độ.
- **Interpretability / Explainability (Khả năng giải thích) - Tầng minh bạch** (*nghĩa là "AI phải giải thích được tại sao nó làm thế"*)
 - **Vấn đề "Hộp đen" (Black Box):** Các mô hình Deep Learning rất giỏi nhưng thường **không ai biết tại sao nó ra kết quả đó**.
 - **Mục tiêu:** AI không được phép là một cái hộp đen bí ẩn. Nó phải trả lời được: "*Tại sao mày từ chối hồ sơ vay vốn của tôi?*" hay "*Tại sao mày chẩn đoán bức ảnh X-quang này là ung thư?*".
 - **Tầm quan trọng:** Nếu **không giải thích được** (Explainable AI - XAI), **con người sẽ không dám tin tưởng để AI ra quyết định** trong y tế, pháp luật, tài chính.
- **Making Decision (Ra quyết định) - Tầng hành động**
 - **Định nghĩa:** Đích đến cuối cùng. Từ việc hiểu và suy luận, AI phải đưa ra **hành động tối ưu hoặc đề xuất hành động** cho con người.
 - **Ví dụ:**
 - *Xe tự lái:* Phát hiện vật cản (Understanding) → Tính toán va chạm (Reasoning) → **Đạp phanh (Decision Making)**.
 - *AI Chứng khoán:* Phân tích tin tức (NLP) → Dự đoán xu hướng → **Đặt lệnh Mua/Bán**.
- Mọi hệ thống AI hiện đại (dù là CV hay NLP) đều đang cố gắng đi theo lộ trình này:



- Mục tiêu của AI không phải là bắt chước con mắt hay cái tai (Perception), mà là bắt chước bộ não (Reasoning & Decision). Và để nuôi bộ não đó, Data chính là thức ăn.

6.3. Some Machine Learning, Deep Learning terms

• Key Differences Between Machine Learning and Deep Learning

Khía cạnh	Machine Learning (ML)	Deep Learning (DL)
Phương pháp huấn luyện	Học có giám sát (Supervised), Không giám sát (Unsupervised), Bán giám sát (Semi-Supervised), Học tăng cường (Reinforcement learning), RLHF (Reinforcement Learning from Human Feedback - Học tăng cường từ phản hồi của con người) ...	DL là một nhánh con của ML để giải quyết các bài toán phức tạp hơn → sử dụng các phương pháp tương tự. Deep Learning là một kỹ thuật trong Machine Learning, chuyên sâu vào việc sử dụng mạng nơ-ron nhiều lớp.
Trích xuất đặc trưng (Feature Extraction)	Yêu cầu trích xuất thủ công → Con người phải tự tay trích xuất các đặc điểm (cạnh, đường cong, cường độ điểm ảnh).	Học các đặc trưng một cách tự động từ dữ liệu thô. Trích xuất theo phân cấp (đường nét → hình dạng → chữ số).
Loại thuật toán	Sử dụng các thuật toán như Cây quyết định (Decision Trees), Random Forests, SVM.	Sử dụng các mạng nơ-ron sâu (CNN, RNN, Transformer).
Yêu cầu dữ liệu	Hoạt động tốt với bộ dữ liệu từ nhỏ đến trung bình .	Yêu cầu bộ dữ liệu lớn để có thể tổng quát hóa tốt.
Sức mạnh tính toán	Có thể chạy trên CPU .	Cần GPU/TPU để xử lý nhanh.
Thời gian huấn luyện	Huấn luyện nhANH hơn , ít phức tạp hơn.	Huấn luyện lâu hơn do có nhiều lớp mạng sâu.
Khả năng giải thích (Interpretability)	Dễ giải thích hơn (ví dụ: cây quyết định hiển thị rõ các quy tắc).	"Hộp đen" (khó giải thích cách nó đưa ra quyết định).
Ứng dụng	Dùng cho dữ liệu có cấu trúc (ví dụ: phát hiện gian lận, hệ thống gợi ý).	Dùng cho dữ liệu phi cấu trúc (ví dụ: hình ảnh, video, xử lý ngôn ngữ tự nhiên - NLP).
Thực hiện tốt về các tác vụ	Tự động hóa, Lặp đi lặp lại → Tập trung nhiều hơn vào tự động hóa với dữ liệu có cấu trúc.	Xử lý dữ liệu phức tạp và phi cấu trúc (ví dụ: hình ảnh, giọng nói, và phân tích thời gian thực).

• Reinforcement Learning (RL)

- Trong học tăng cường, một tác nhân (agent) học cách đưa ra quyết định dựa trên các hành động mà nó thực hiện trong môi trường, nhằm tối đa hóa tổng phần thưởng (reward).
- AI học bằng cách **thử và sai, sử dụng phần thưởng (reward) hoặc hình phạt (penalty)**.
- Quy trình:** Tác nhân thực hiện một hành động trong môi trường → **Môi trường** phản hồi lại với một kết quả (state) mới và một phần thưởng (reward) → **Tác nhân** sử dụng phần thưởng đó để cải thiện chiến lược (policy) của mình trong các lần tương tác sau.

- Trong các tác vụ RL truyền thống, phần thưởng thường được xác định trước (hard-coded) hoặc có sẵn trong môi trường, và tác nhân học cách tối đa hóa phần thưởng đó. Tuy nhiên, trong nhiều trường hợp thực tế, **việc xác định hàm phần thưởng chính xác là rất khó khăn, đặc biệt trong các tác vụ phức tạp**, nơi có sự tham gia của con người, như trò chuyện, đánh giá nội dung, hoặc các quyết định đòi hỏi nhận thức tinh tế.
- **Reinforcement Learning from Human Feedback (RLHF)**
 - Thay vì chỉ dựa vào các phần thưởng tự động hoặc cố định từ môi trường, **con người tham gia vào quá trình huấn luyện và cung cấp phản hồi cho mô hình**. Phản hồi này có thể ở dạng:
 - **Phản hồi gián tiếp**: Con người đánh giá hành động của tác nhân và cung cấp phản hồi (ví dụ, “hành động này tốt” hoặc “hành động này sai”).
 - **Phản hồi trực tiếp**: Con người có thể hướng dẫn tác nhân trong từng bước bằng cách chỉ ra các hành động đúng/sai hoặc điều chỉnh mô hình theo các tiêu chí đạo đức, xã hội.
 - **Quy trình hoạt động của RLHF trong thực tế**:
 - **1. Thu thập Phản hồi từ Con người (Feedback Collection)**
 - Một **team** (nhóm) sẽ thực hiện nhiệm vụ **thu thập phản hồi từ con người, đặc biệt là các phản hồi tiêu cực** (negative feedback) nếu mô hình đưa ra những hành động sai hoặc không mong muốn.
 - Phản hồi này có thể được thu thập theo nhiều cách, ví dụ: người dùng cung cấp đánh giá về chất lượng câu trả lời, hoặc đội ngũ kiểm thử người có thể trực tiếp phản hồi về hành vi của mô hình.
 - Quan trọng là trong nhiều trường hợp, không chỉ phản hồi tiêu cực mà cả phản hồi tích cực cũng cần được thu thập để đánh giá mô hình một cách toàn diện.
 - **2. Đánh giá và Phân tích Phản hồi (Feedback Evaluation)**
 - Sau khi thu thập được phản hồi từ con người, nhóm kỹ sư hoặc các chuyên gia sẽ **đánh giá và phân tích** phản hồi đó để xác định liệu mô hình có đang làm đúng hay không.
 - Phản hồi có thể bao gồm việc đánh giá các hành động của mô hình trong các tình huống cụ thể và kiểm tra xem hành động của mô hình có đáp ứng đúng yêu cầu hay giá trị của người dùng hay không.
 - Phản hồi này có thể được gắn với **một điểm số hoặc một giá trị** nào đó để mô hình có thể học từ đó.
 - **3. Cập nhật và Huấn luyện lại Mô hình (Model Update and Retraining)**
 - Sau khi đánh giá phản hồi, thông tin này sẽ được **backpropagate (truyền ngược lại) vào hệ thống để cập nhật mô hình**.
 - Mô hình sẽ điều chỉnh chính sách hành động của mình dựa trên các phản hồi từ con người.
 - Mô hình có thể học cách tránh các hành động tiêu cực và tối ưu hóa các hành động tích cực dựa trên phản hồi đã nhận.

- **4. Lặp lại Quy trình (Iterative Process)**
 - Mỗi lần mô hình được cập nhật, các phản hồi từ con người sẽ tiếp tục được thu thập và giúp cải thiện mô hình.
 - Nhóm sẽ tiếp tục thu thập phản hồi mới, phân tích, cập nhật mô hình và kiểm tra kết quả. Mỗi chu kỳ giúp mô hình ngày càng trở nên chính xác và hiệu quả hơn trong việc thực hiện các nhiệm vụ.
- **5. Tự Động Hóa và Quy Mô:** Với các ứng dụng quy mô lớn (như GPT hoặc các hệ thống tự động khác), quá trình này có thể được **tự động hóa** một phần để tối ưu hóa việc thu thập phản hồi, phân tích và cập nhật mô hình. Tuy nhiên, con người vẫn đóng vai trò quan trọng trong việc đảm bảo rằng phản hồi được đánh giá chính xác và có ý nghĩa.
- **RLHF đặc biệt hữu ích trong các tình huống:**
 - **Hàm phần thưởng khó xác định:** Khi không thể lập trình một hàm phần thưởng cụ thể, nhưng con người có thể đánh giá kết quả của mô hình.
 - **Chất lượng yêu cầu cao:** Ví dụ trong việc huấn luyện các mô hình trò chuyện (chatbots) như GPT, nơi việc hiểu ngữ cảnh, các yếu tố đạo đức và xã hội rất quan trọng, và con người có thể cung cấp phản hồi trực tiếp để đảm bảo hành vi của mô hình là phù hợp.
 - **Các tác vụ phức tạp và mở rộng:** Ví dụ, các trò chơi hoặc môi trường với số lượng trạng thái và hành động quá lớn để mô hình có thể khám phá một cách đầy đủ.
- **Ví dụ về RLHF trong thực tế**
 - **Các mô hình ngôn ngữ:** Trong quá trình huấn luyện các mô hình, phản hồi từ con người được sử dụng để giúp mô hình hiểu rõ hơn về ngữ nghĩa, tính hợp lý của câu trả lời và các yếu tố xã hội/đạo đức khi giao tiếp với người dùng.
 - **Xe tự lái:** Xe tự lái có thể sử dụng RLHF để học từ các phản hồi của người lái thử hoặc người giám sát trong quá trình huấn luyện, giúp xe điều chỉnh hành vi của mình để phù hợp hơn với môi trường thực tế.

6.4. Các công cụ & thư viện của Python & hỗ trợ image processing

- **Ưu điểm của Python:** Python có các hàm viết sẵn và có thể tinh chỉnh (fine-tune).
- **Kernel Python:** Quy trình chạy mã Python bên trong Jupyter Notebook.
- Lựa chọn công cụ để thực hiện các tác vụ AI

Scenario	Best Choice
Quick experiments, data analysis, and ML	Jupyter Notebook
Large projects, software development, debugging	VSCode
Data science & AI with pre-installed libraries	Anaconda

- **utils:** Thư mục lưu trữ các "phương thức hỗ trợ" trong trường hợp cần tái sử dụng hàm (vẽ biểu đồ, đọc dữ liệu,...).

- **Virtual Environment (Môi trường ảo)**

- Một không gian cách ly, giúp bạn cài đặt các gói Python riêng biệt mà không ảnh hưởng đến cài đặt Python toàn cục (global). Mỗi môi trường ảo có thể coi như một **trình thông dịch Python riêng biệt** với các thư viện và phiên bản cài đặt riêng biệt.
- **Không ảnh hưởng đến môi trường toàn cục:** Các thay đổi trong môi trường ảo sẽ không ảnh hưởng đến các cài đặt Python hoặc thư viện toàn cục của hệ thống.
- **Cài đặt độc lập cho mỗi dự án:** Mỗi môi trường ảo có thể có các thư viện, phiên bản riêng biệt, giúp tránh xung đột khi làm việc với nhiều dự án khác nhau.
- **Có thể tạo nhiều môi trường ảo:** Bạn có thể tạo nhiều môi trường ảo cho các dự án khác nhau và dễ dàng chuyển đổi giữa chúng mà không gặp vấn đề xung đột giữa các thư viện hoặc phiên bản.
- **Khi nào nên sử dụng môi trường ảo?**
 - Khi làm việc trên nhiều dự án có các thư viện và phiên bản khác nhau.
 - Khi triển khai (deploy) ứng dụng.
 - Khi làm việc với các framework như Django, Flask, hoặc các thư viện AI/ML.
- **Cách tạo và quản lý môi trường ảo trên Windows**
 - Để tạo môi trường ảo, bạn mở **Command Prompt** hoặc **PowerShell**, sau đó gõ lệnh sau: `python -m venv .venv`
.venv là tên thư mục sẽ chứa môi trường ảo. Bạn có thể thay đổi tên này theo ý muốn.
Trên một số máy cài cả Python 2 và Python 3, hoặc cài qua Windows Store, đôi khi lệnh python không trỏ đúng vào phiên bản bạn muốn. An toàn nhất nên dùng: `py -m venv .venv` hoặc `python3 -m venv .venv` (nếu dùng Git Bash/WSL).
 - **Kích hoạt (activate) môi trường ảo**
 - Sau khi tạo môi trường ảo, bạn cần kích hoạt nó để bắt đầu sử dụng. Gõ lệnh sau trong **PowerShell** hoặc **Command Prompt**:
`.venv\Scripts\Activate`
 - Sau khi kích hoạt, bạn sẽ thấy tên môi trường ảo (ví dụ: .venv) hiển thị trong dấu nhắc lệnh, cho biết bạn đang sử dụng môi trường ảo.
 - **Hủy kích hoạt (deactivate) môi trường ảo:** Khi bạn muốn thoát khỏi môi trường ảo và quay lại môi trường Python toàn cục, gõ lệnh: `deactivate`
 - **Đổi tên môi trường ảo** Nếu bạn muốn thay đổi tên của môi trường ảo, bạn có thể dùng lệnh sau trong **PowerShell**: `Rename-Item .venv my_env`
.venv là tên cũ, và my_env là tên mới bạn muốn đặt.
 - **Xóa môi trường ảo:** Để xóa môi trường ảo, bạn dùng lệnh sau: `Remove-Item -Recurse -Force .venv` → Lệnh này sẽ xóa thư mục môi trường ảo và tất cả các tệp bên trong.
 - **Xóa môi trường ảo khỏi danh sách Python Interpreter:** Nếu bạn đã xóa môi trường ảo nhưng nó vẫn xuất hiện trong danh sách các trình thông dịch (interpreter) trên **VS Code** hoặc **IDE** khác, bạn có thể: **Nhấn Ctrl + Shift + P, tìm và chọn Python: Select Interpreter**, sau đó **xóa** môi trường ảo đã không còn sử dụng.

- **Sửa lỗi chính sách PowerShell khi kích hoạt môi trường ảo:** PowerShell có một chính sách bảo mật ngăn không cho chạy các script chưa được ký. Nếu gặp lỗi khi kích hoạt môi trường ảo, bạn cần thay đổi chính sách thực thi (execution policy). Để làm điều này, bạn có hai cách:
 - **Chỉnh sửa tạm thời (mỗi lần mở PowerShell):** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
 - **Chỉnh sửa vĩnh viễn (một lần duy nhất):** `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`
- **Kiểm tra các thư viện đã cài đặt:** Để xem danh sách các thư viện (packages) đã cài đặt trong môi trường ảo, bạn dùng lệnh: `pip list`
- **NumPy:** Thư viện trả về **cấu trúc dữ liệu mảng N-chiều** trong Python, hỗ trợ tốt nhất cho **đại số tuyến tính** trong AI, là cốt lõi của các phép toán ma trận và vector trong AI.
 - **Các tính năng nổi bật của NumPy:**
 - **Mảng N-chiều (NumPy Arrays):** Mảng NumPy có thể là **mảng một chiều** (vector), **mảng hai chiều** (ma trận) hoặc **mảng nhiều chiều** (N-dimensional array). Mảng này được thiết kế để thay thế cho danh sách Python, giúp xử lý dữ liệu lớn hiệu quả hơn.
 - **Tính toán Ma trận và Vector:** NumPy hỗ trợ các phép toán **ma trận** và **vector** cực kỳ nhanh chóng và dễ dàng, ví dụ như cộng, nhân ma trận, phép nhân giữa ma trận và vector, v.v.
 - **Các phép toán số học (Arithmetic Operations):** NumPy hỗ trợ phép toán số học như cộng, trừ, nhân, chia, và tính toán các phép toán phức tạp hơn trên mảng một cách dễ dàng.
 - **Hỗ trợ các phép toán thống kê và phân tích:** NumPy cung cấp các hàm thống kê và phân tích số liệu như tính trung bình, phương sai, độ lệch chuẩn, v.v.
 - **Xử lý dữ liệu lớn:** NumPy có thể xử lý rất nhanh các mảng dữ liệu lớn mà không tốn quá nhiều bộ nhớ, điều này cực kỳ quan trọng trong **Phân tích dữ liệu lớn (Big Data Analysis)**.
 - **Ứng dụng của NumPy trong AI và các lĩnh vực khác:**
 - **Deep Learning:** Trong Deep Learning, các phép toán trên ma trận và vector đóng vai trò quan trọng. Các mô hình học sâu như **neural networks** phụ thuộc rất nhiều vào các phép toán ma trận, đặc biệt trong việc tính toán đầu vào, đầu ra và gradient trong quá trình huấn luyện. NumPy giúp việc tính toán này trở nên nhanh chóng và hiệu quả. Ví dụ, trong việc tính toán gradient cho việc tối ưu hóa trọng số trong mạng nơ-ron, NumPy được sử dụng để xử lý các phép toán ma trận.
 - **Machine Learning:** Trong Machine Learning, NumPy hỗ trợ việc tính toán các **phương sai**, **hiệu số** giữa các điểm dữ liệu, **tính toán hàm chi phí (cost function)**, và các phép toán liên quan đến **linear regression** hay **classification**. Đặc biệt, trong quá trình huấn luyện, các phép toán ma trận giúp việc tối ưu hóa mô hình trở nên nhanh chóng và dễ dàng.

- **Xử lý ảnh (Image Processing):** Trong **xử lý ảnh**, ảnh có thể được biểu diễn dưới dạng **mảng ma trận** (2D hoặc 3D) và các phép toán như **lọc ảnh**, **biến đổi ảnh**, **phát hiện biên**, v.v., có thể dễ dàng thực hiện bằng NumPy. Ví dụ, ảnh có thể được lưu dưới dạng một mảng NumPy, nơi mỗi pixel của ảnh là một giá trị trong mảng. Các phép toán như **mở rộng**, **thu nhỏ**, hoặc **biến đổi màu sắc** có thể được thực hiện trực tiếp trên mảng này.
- **Phân tích Dữ liệu:** NumPy cũng là nền tảng của các thư viện phân tích dữ liệu phổ biến như **Pandas**, giúp xử lý và phân tích các tập dữ liệu lớn (Big Data). Các phép toán thống kê cơ bản như **tính trung bình**, **độ lệch chuẩn**, **phân phối dữ liệu** có thể được thực hiện dễ dàng với NumPy.
- **ucimlrepo:** Thư viện để truy cập bộ dữ liệu từ UCI.
- **Pandas - DA (Thao tác dữ liệu):** Xử lý dữ liệu có cấu trúc (bảng, CSV, Excel) một cách hiệu quả, giúp trực quan hóa. Hầu hết trả về kiểu dữ liệu bằng "DataFrame", dùng cho SQL,...
- **Matplotlib & Seaborn:**
 - **Để vẽ các biểu đồ cơ bản** và tùy chỉnh, cần tinh chỉnh sâu và code ở mức thấp (low level) → **Matplotlib** được ưu tiên.
 - Để **phân tích dữ liệu thống kê & trực quan hóa** nhanh → **Seaborn** thường được chọn.
 - Seaborn cho trực quan hóa cấp cao và Matplotlib để tinh chỉnh chi tiết.
- **SciPy:** Tính toán khoa học và số học nâng cao.
- **Scikit-Learn (Machine Learning):** Huấn luyện các mô hình ML như hồi quy, phân loại và phân cụm.
- **TensorFlow & PyTorch (Deep Learning):** Xây dựng và huấn luyện mạng nơ-ron (khuyến dùng **PyTorch**).
- **OpenCV - Pillow:** Các thư viện cho xử lý ảnh.
 - **Pillow** là một thư viện mở rộng của **PIL** (Python Imaging Library), được sử dụng rộng rãi để thao tác với hình ảnh trong Python. Nó cung cấp các chức năng cơ bản như đọc, viết và xử lý ảnh với nhiều định dạng khác nhau (JPEG, PNG, BMP, GIF, v.v.).
 - **Các tính năng của Pillow:**
 - **Chuyển đổi ảnh thành mảng NumPy:** Pillow cho phép bạn dễ dàng chuyển đổi ảnh thành mảng **NumPy array** để có thể sử dụng trong các phép toán số học. **Định dạng mặc định của các ảnh khi sử dụng Pillow là RGB (Red, Green, Blue).**
 - **Mở và Lưu ảnh:** Bạn có thể dễ dàng mở ảnh từ tệp và lưu ảnh với các định dạng khác nhau.
 - **Chỉnh sửa ảnh:** Pillow hỗ trợ các thao tác cơ bản như thay đổi kích thước, cắt xén, xoay, áp dụng bộ lọc, v.v.

- Ví dụ chuyển ảnh sang mảng NumPy với Pillow:

```
from PIL import Image
import numpy as np

# Mở ảnh bằng Pillow
img = Image.open("image.jpg")

# Chuyển ảnh sang mảng NumPy
img_array = np.array(img)

print(img_array.shape)
# Output: (height, width, 3), tức là ảnh RGB (3 kênh màu)
```

- **OpenCV** là một thư viện mạnh mẽ và phổ biến hơn khi nói đến **thị giác máy tính (computer vision)** và xử lý ảnh trong thời gian thực. Nó hỗ trợ các phép toán xử lý ảnh phức tạp như nhận diện đối tượng, phát hiện biên, theo dõi chuyển động, và xử lý video.

- **Các tính năng của OpenCV:**

- **Chuyển ảnh thành mảng NumPy:** OpenCV cũng trả về ảnh dưới dạng mảng **NumPy array**, nhưng định dạng ảnh mà OpenCV sử dụng là **BGR (Blue, Green, Red)** thay vì **RGB** như trong Pillow → nếu dùng thì cần chuyển đổi (convert) về **RGB**.
- **Đọc và hiển thị ảnh:** OpenCV có thể đọc ảnh từ các file hình ảnh và hiển thị chúng trực tiếp trong cửa sổ GUI.
- **Xử lý ảnh phức tạp:** OpenCV cung cấp hàng loạt các hàm để thực hiện các phép toán xử lý ảnh phức tạp, bao gồm các phép toán biến đổi Fourier, lọc ảnh, nhận dạng đối tượng, v.v.

- Ví dụ chuyển ảnh từ BGR (OpenCV) sang RGB (Pillow):

```
import cv2
import numpy as np

# Đọc ảnh bằng OpenCV
img_bgr = cv2.imread("image.jpg")

# Chuyển ảnh từ BGR sang RGB
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Hiển thị ảnh bằng OpenCV (dành cho BGR)
cv2.imshow("BGR Image", img_bgr)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- Trong ví dụ trên, ta cần chuyển đổi **BGR** sang **RGB** khi làm việc với OpenCV và Pillow cùng lúc vì OpenCV đọc ảnh theo chuẩn BGR, trong khi Pillow sử dụng chuẩn RGB.
- NumPy hỗ trợ nhiều kiểu dữ liệu khác nhau, không chỉ **uint8**. Các kiểu dữ liệu này có thể là **số nguyên** (integer), **số thực** (float), **số phức** (complex), hoặc các loại dữ liệu khác. **Nên trả ảnh về dạng numpy.array với kiểu dữ liệu là uint8**
 - **Unsigned Integer 8** (Số nguyên không dấu 8-bit), giá trị từ 0 → 255: $2^8 = 256$ giá trị.
 - Điều này rất phù hợp khi bạn xử lý ảnh có màu sắc **RGB**, vì mỗi kênh màu (Red, Green, Blue) có thể sử dụng **một byte (8-bit)** để biểu diễn giá trị từ 0 đến 255.

```
# Tạo một mảng NumPy với kiểu dữ liệu uint8  
arr = np.array([0, 128, 255], dtype=np.uint8)
```

Danh mục tài liệu sử dụng:

- **Tài liệu học phần:** TS. Nguyễn Thị Khánh Tiên, *Chương 1: Tổng quan về thị giác máy tính và xử lý ảnh*, Slide Học phần Xử lý ảnh & Thị giác máy tính, Viện Công Nghệ Thông Tin, Điện, Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
- **Tài liệu tham khảo:**
 - Trần Lê Anh (3/2025), *Chapter 1: INTRODUCTION*, Slide Image Processing and Computer Vision, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
 - Trần Đăng Nam, *Lecture Note Computer Vision*.
 - Gà Lại Lập Trình (12/2021), *Bài 1: Cài đặt opencv python và truy xuất ảnh - opencv python cho người mới - Install opencv python*, https://www.youtube.com/watch?v=fR9Wx8D_1LA
 - Các công cụ AI hỗ trợ.